

Collective-Aware Router Microarchitecture in gem5 Garnet

Multicast, express-link bypass, and tensor all-reduce

Chunyu Liu and Boyan Pu

September 7, 2026

Abstract

GPU training repeatedly maps gradient tensors and synchronization collectives onto the network. We study how preserving this communication structure in gem5 Garnet changes traffic, latency, and throughput. The implementation covers the two mechanisms required by Lab 4 Topic 3—physical bypass links and one-to-many multicast—and evaluates each against an appropriate baseline. On 4×4 , 162 matched cases show 39.51% mean internal-link flit reduction, $3.638 \times$ geometric-mean latency speedup, and 190.89% mean throughput improvement relative to replicated unicast. On 8×8 , the same fanouts show 34.95% traffic reduction, $3.472 \times$ latency speedup, and 208.34% throughput improvement across another 162 cases, with no throughput regressions. A further 108-case 8×8 scale-out study reaches 75.39% traffic reduction at fanout 64. The ten observed regressions are confined to low-fanout 4×4 cases and arise from atomic branch synchronization. The bypass study compares distance-scaled diagonal and stride-2 express-link arms with offline multi-hop route tables and conservative runtime admission. Under the packet-averaged traffic-pattern aggregation, diagonal reaches $1.426 \times$ and $+4.27\%$ on 4×4 , $1.165 \times$ and $+12.90\%$ on 8×8 ; stride-2 reaches $1.312 \times$ and $+4.37\%$ on 4×4 , and $1.620 \times$ and $+39.37\%$ on 8×8 . The plain-Mesh baseline has fewer links, ports, and buffers, so the comparison measures the performance of the augmented topology at its stated resource cost. A Topic 4 case study maps 15 scaled eight-H100 all-reduce events to multi-flit collective requests and shortens the simulated replay window by $4.728 \times$ relative to 6,720 serialized scalar-lane rounds, with identical source and Router-forwarded flit counts. Code and experimental artifacts are available at <https://github.com/Haru-LCY/ai-x>.

1 Introduction

Large-scale GPU training alternates local computation with global communication. During each iteration, GPU ranks produce gradient tensors, exchange them through collectives such as all-reduce and broadcast, and wait for the synchronized result before beginning the next step. As illustrated in Figure 1, this repeated dependency maps structured tensor operations onto packets and flits in the interconnection network. Network behavior therefore contributes directly to the synchronization path of training.

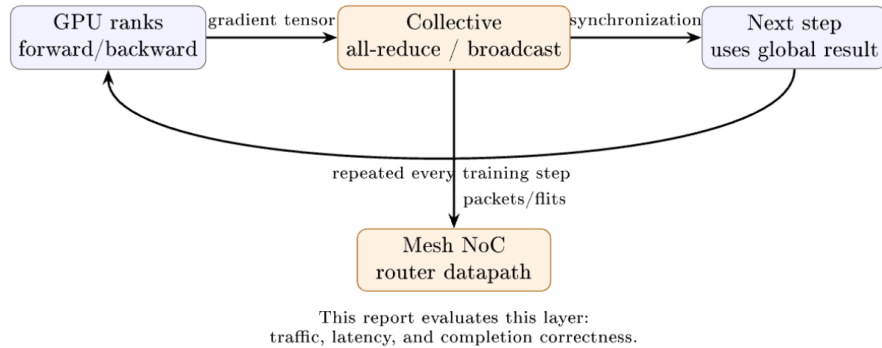


Figure 1: GPU training repeatedly maps gradient tensors and collective synchronization onto the Mesh NoC. This work studies how the Router datapath represents and transports that structure.

NCCL exposes the participant groups and operations associated with broadcast and all-reduce [7], but a conventional network reduces them to independent packets and destinations. The three mechanisms in this report preserve

or exploit structure at complementary levels. Tree multicast shares common route prefixes for one-to-many delivery. Tensor streaming retains an all-reduce event as a multi-flit collective request rather than serializing its lanes into thousands of independent rounds. Express links shorten selected paths through the underlying Mesh.

This organization also maps directly to the project specification. Lab 4 Topic 3 requires both Bypass and Multicast, together with appropriate baselines. We compare physical express-link bypass with plain Mesh XY and Router-level tree multicast with replicated unicast on the same Mesh. Topic 4 includes trace-based traffic, which we address by replaying measured H100 collectives through the tensor datapath. The evaluation asks:

1. Can Router-side replication reduce physical multicast traffic without changing the delivered destination set?
2. Do express links still improve latency on 4×4 and 8×8 after their span-dependent wire delay, added radix, and congestion are counted, and how does the result change with scale?
3. Can a traced all-reduce remain a multi-flit tensor request instead of thousands of serialized scalar-lane rounds?

Tree multicast shares common route prefixes and produces its largest traffic reduction at high fanout. Atomic fanout couples branch progress and can reduce throughput when one output is congested. The multi-hop, pressure-aware bypass primarily benefits loaded structured traffic. Event-level tensor packetization then carries the GPU-training narrative from group delivery to the representation and execution of complete all-reduce tensors.

Each result uses its corresponding unit: internal-link flit reduction, paired latency ratio, logical throughput change, or trace window. We report geometric means for latency ratios, arithmetic means for additive changes, medians and extrema for skewed distributions, and the number of regressions. Three deterministic seeds are retained as paired observations rather than collapsed before ratio calculation.

Section 2 introduces the shared Garnet substrate and metrics. Sections 3 and 4 present multicast and bypass, respectively, each from design and implementation through validation and evaluation. Section 5 presents the trace-driven H100 tensor study. The final sections discuss cross-mechanism scope, authorship, and reproducibility.

2 Background and common methodology

2.1 Relation to prior work

Garnet provides the detailed cycle-level network substrate used here [2]. Virtual Circuit Tree Multicasting established Router-level tree replication as an alternative to repeated unicast [3]; our implementation instead uses a compact destination bitmap and atomic branch allocation. Express Virtual Channels bypass intermediate Router pipelines over conventional channels [4]; our mechanism instead installs explicit span-2 physical links and therefore pays added port, buffer, and wire proxies. For routing safety, the offline oracle applies the channel-dependency condition of Dally and Seitz [5] to its generated static paths. SHARP demonstrates hardware streaming aggregation over reduction trees [6]; the tensor extension applies the same broad principle to lane-tagged Garnet packets and evaluates the effect of preserving event-level request structure.

The implementation therefore covers both Topic 3 mechanisms and adds one Topic 4 trace-based extension. The following definitions specify the baseline and measurement procedure for each comparison.

2.2 Common experimental methodology

The implementation is based on gem5 v23.0.0.1 [1] and the cycle-level Garnet model [2]. Plain Mesh and collective trees use deterministic X-first XY routing. Evaluated revisions and source hashes are listed in Section 7.

Each performance comparison is paired: baseline and proposed runs use the same topology, workload realization, packet size, load, and seed. Multicast uses replicated unicast as its baseline, bypass uses plain Mesh XY, and the tensor study compares scalar-lane and event-level representations of the same trace. Mechanism-specific configurations and validation are stated in their own sections rather than combined into one global parameter table.

Latency is reported as baseline latency divided by proposed latency, so a value above one indicates a speedup. Internal-link traffic and throughput are reported as percentage changes from their paired baselines. A multicast request completes only after every selected destination receives the complete packet; multicast throughput therefore counts completed logical requests per cycle. Bypass throughput instead counts completed flits per node per source cycle.

We use arithmetic means for percentage changes and geometric means for latency ratios within each directly paired group. For the bypass headline, the four packet-size geometric means are averaged arithmetically within each traffic pattern, and the four traffic-pattern values are then combined geometrically; Table 2 states this order explicitly. Full result-set locations are documented in Section 7.

3 Tree multicast

3.1 Design and baseline

The baseline Network Interface injects one physical packet per remote destination and records a selected local destination directly. The multicast path injects one packet carrying a 64-bit destination bitmap. Each Router computes the required pruned-tree child branches from that bitmap and makes a local copy only where selected. Head, body, and tail state is retained per branch. Fanout is atomic: a flit advances only when all selected output VCs have credit. This preserves exact multi-flit delivery, but it also couples a fast branch to a blocked branch—an important interpretation of the throughput results below. The bitmap bounds one collective domain to 64 Routers.

Figure 2 contrasts replicated unicast with the shared-prefix tree used by the proposed mechanism.

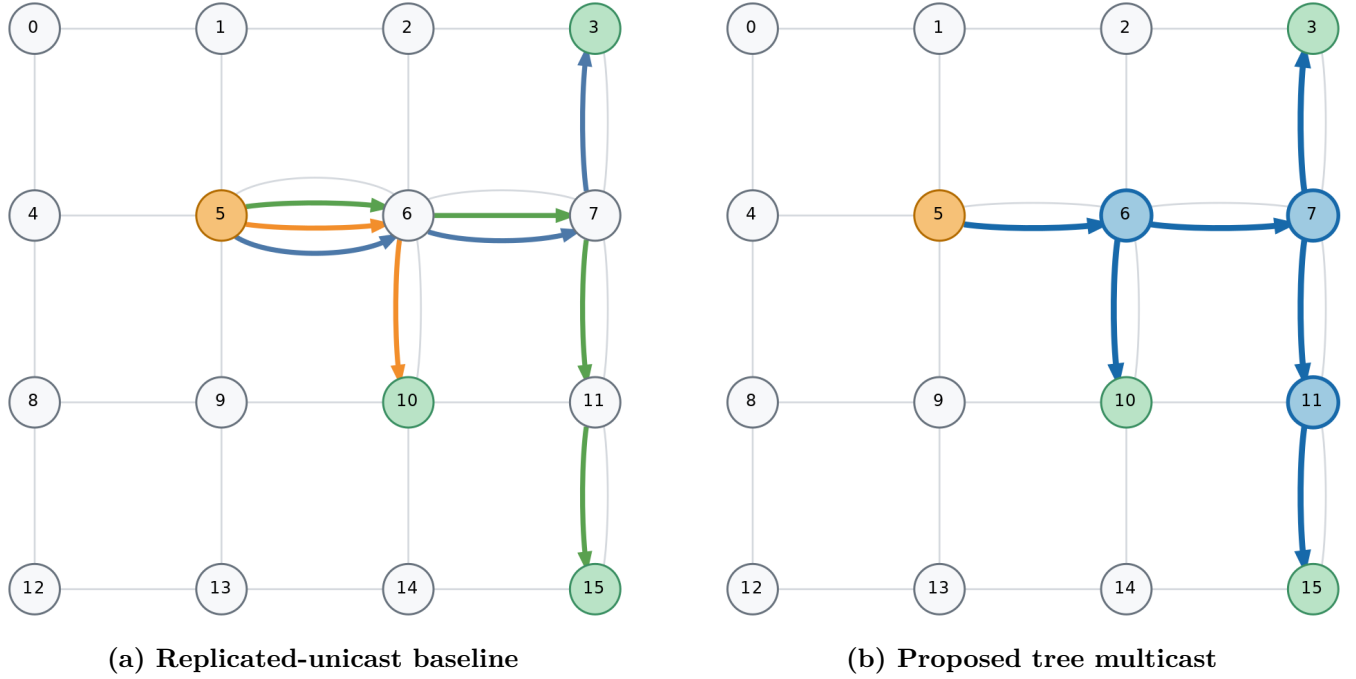


Figure 2: Illustrative 4x4 multicast baseline and proposed path.

3.2 Garnet implementation

The head flit first computes the pruned-tree children selected by its bitmap. The Router commits branch state only if every required output has a free VC; body and tail flits reuse those VCs. Listing 1 is an excerpt from the implemented two-phase allocation. This makes a multi-flit packet exact and prevents partial branch allocation, but also explains the measured head-of-line coupling: one blocked branch stalls the whole fanout.

Listing 1: Source excerpt: atomic multicast branch allocation (Router.cc).

```
const int vnet = head_flit->get_vnet();
for (const int destination : destinations) {
    const int output = collectiveOutputForChild(destination);
    if (output < 0 || !m_output_unit[output]->has_free_vc(vnet)) {
        m_network_ptr->recordMulticastCreditStall();
        return false;
    }
}
for (const int destination : destinations) {
    const int output = collectiveOutputForChild(destination);
    const int outvc = m_output_unit[output]->select_free_vc(vnet);
    fatal_if(outvc < 0, "Router %d lost multicast VC allocation", m_id);
    m_multicast_branches.push_back({output, outvc, destination});
}
```

Table 1: Multicast results over the fanouts listed for each Mesh size. Latency is the geometric mean; internal-link flit reduction and throughput change are arithmetic means over all matched cases included in each row.

Mesh	Fanouts	Latency speedup	Link-flit reduction	Throughput change
4×4	4, 8, 16	3.638×	39.51%	+190.89%
8×8	4, 8, 16, 32, 64	6.331×	48.53%	+681.77%

3.3 Experimental setup and validation

For every multicast pair, the baseline and tree runs use the same Mesh, destination set, 1/4/16-flit packet size, request probability 0.1/0.5/1.0, optional uniform background load 0/0.1, and seed 1, 7, or 17. The balanced matrix uses source Router 5 on 4×4 and Router 27 on 8×8 with fanouts 4, 8, and 16; the separate 8×8 scale-out adds fanouts 32 and 64. Each run uses 20 warm-up, 100 measured, and 20 cooldown requests. Destination sets below full fanout are sampled without replacement and may include the source Router.

Correctness validation covers payload values, delivery sets, flit ordering, and completion under backpressure. The functional multicast matrix contains 30 destination/topology cases from 2×2 through 8×8, two implementations, and four packet sizes: 240 executions, or 120 mode-matched comparisons. The performance study contains 324 balanced cross-topology pairs and 108 additional 8×8 scale-out pairs. Analysis scripts check pair completeness, finite metrics, delivery metadata, and run counts before generating aggregate figures. All 240 executions match the destination bitmap and flit order without missing or duplicate delivery.

3.4 Results on a 4-by-4 Mesh

Tree multicast reduces duplicated physical-link traversal, while its throughput depends on synchronization among output branches. We therefore report link traffic and logical-request throughput separately.

The 4×4 study contains 162 matched cases at fanouts 4, 8, and 16. Tree multicast reduces internal-link flits by 39.51% on average (median 41.18%, range 16.67–53.13%) and achieves 3.638× geometric-mean latency speedup (median 3.588×, range 1.200–22.512×). Mean throughput improves by 190.89%, with a median improvement of 121.66%.

Ten 4×4 cases regress in throughput, all at fanout 4; the worst change is −18.24%. These cases occur when longer packets and higher load make one blocked output hold the other tree branches. The tree still performs less link work, but atomic fanout couples its progress to the slowest branch.

3.5 Results on an 8-by-8 Mesh

The 8×8 study contains 270 matched cases across fanouts 4, 8, 16, 32, and 64. Across this complete set, tree multicast reduces internal-link flits by 48.53% on average, achieves 6.331× geometric-mean latency speedup, and improves mean throughput by 681.77%.

The common-fanout subset contains 162 cases at fanouts 4, 8, and 16. It reduces internal-link flits by 34.95% on average (median 29.63%, range 6.67–52.94%) and achieves 3.472× geometric-mean latency speedup (median 2.667×, range 1.161–20.163×). Mean throughput improves by 208.34%, with a median improvement of 145.39%.

The 8×8 scale-out study adds 108 cases at fanouts 32 and 64. Mean traffic reduction is 62.41% at fanout 32 and 75.39% at fanout 64. Their geometric-mean latency speedups are 11.25× and 21.62×, while mean throughput improves by 895.54% and 1,888.29%, respectively. The larger groups expose more shared route prefixes, so Router-side replication removes an increasing fraction of repeated link traversal.

3.6 Analysis across topology, fanout, and packet size

Topology size is best compared over the common fanouts 4, 8, and 16. Under that matched scope, 4×4 and 8×8 have similar geometric-mean latency speedups (3.638× and 3.472×), while 4×4 removes more link traffic (39.51% versus 34.95%) and 8×8 has the larger mean throughput change (+208.34% versus +190.89%). A fixed fanout occupies a larger fraction of the smaller Mesh, so its destinations more often share the same XY prefixes; on the larger Mesh, the replicated-unicast baseline also pays for longer routes, making source-side copy removal more valuable to throughput. The 8×8 full-range aggregate in Table 1 is higher because it additionally contains fanouts 32 and 64, not because Mesh size alone produces the increase.

Fanout is the dominant structural variable. On 4×4, increasing fanout from 4 to 16 raises geometric-mean latency speedup from 1.824× to 6.892×, link-flit reduction from 26.19% to 53.12%, and mean throughput change from +34.38%

to +383.42%. On 8×8 , the same trends continue from fanout 4 to 64: latency speedup rises from $1.861\times$ to $21.618\times$, link-flit reduction from 29.13% to 75.39%, and throughput change from +44.57% to +1,888.29%. Replicated unicast injects one physical packet per destination, including repeated traversal of shared route prefixes; tree multicast injects one packet and copies it only at branch points. Larger fanout therefore increases both the number of source-generated copies removed and the fraction of link traversal shared by the tree.

Packet size separates physical-work savings from timing effects. For a fixed destination tree, baseline and tree link traversals both scale linearly with the number of flits, so the reduction percentage is unchanged across 1-, 4-, and 16-flit packets in Figure 4. Latency instead peaks at 4 flits: over common fanouts, its geometric-mean speedup is $2.414\times/5.136\times/3.884\times$ on 4×4 and $1.965\times/5.142\times/4.141\times$ on 8×8 for 1/4/16 flits. One-flit packets offer little repeated payload to amortize, whereas 16-flit packets keep branch VCs reserved until the tail and make atomic fanout wait longer for the slowest output; four flits retain substantial sharing without the full long-packet hold time. The same reservation effect makes mean throughput change decrease with packet size: +261.65%/+169.45%/+141.57% on 4×4 and +276.71%/+188.30%/+160.01% on the common 8×8 fanouts. Figures 3–5 therefore show that fanout controls how much network work can be shared, while packet length controls how efficiently the atomic tree can realize that structural saving.

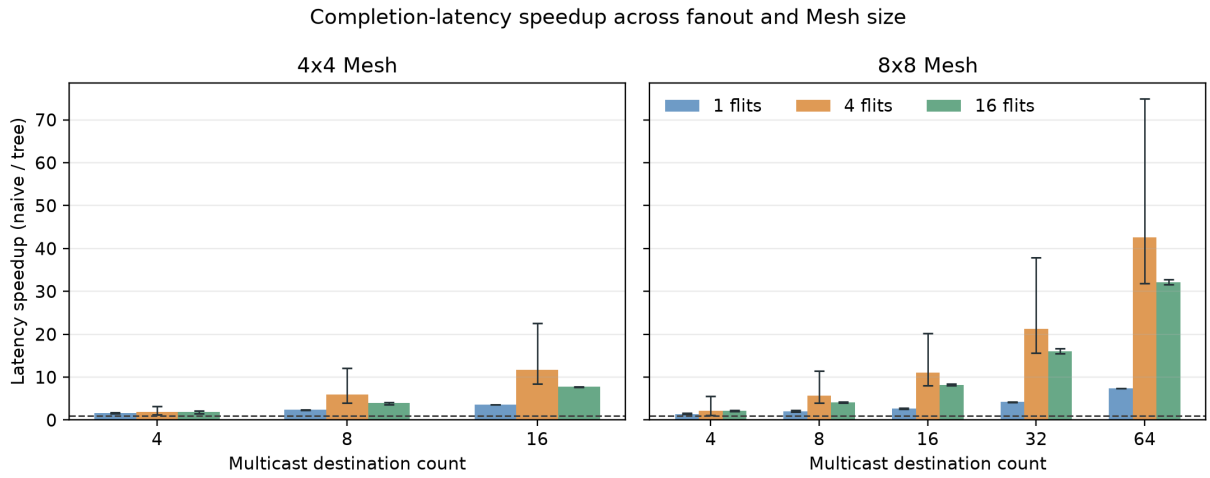


Figure 3: Multicast latency speedup by topology, fanout, and packet size. Bars are geometric means; whiskers show observed ranges.

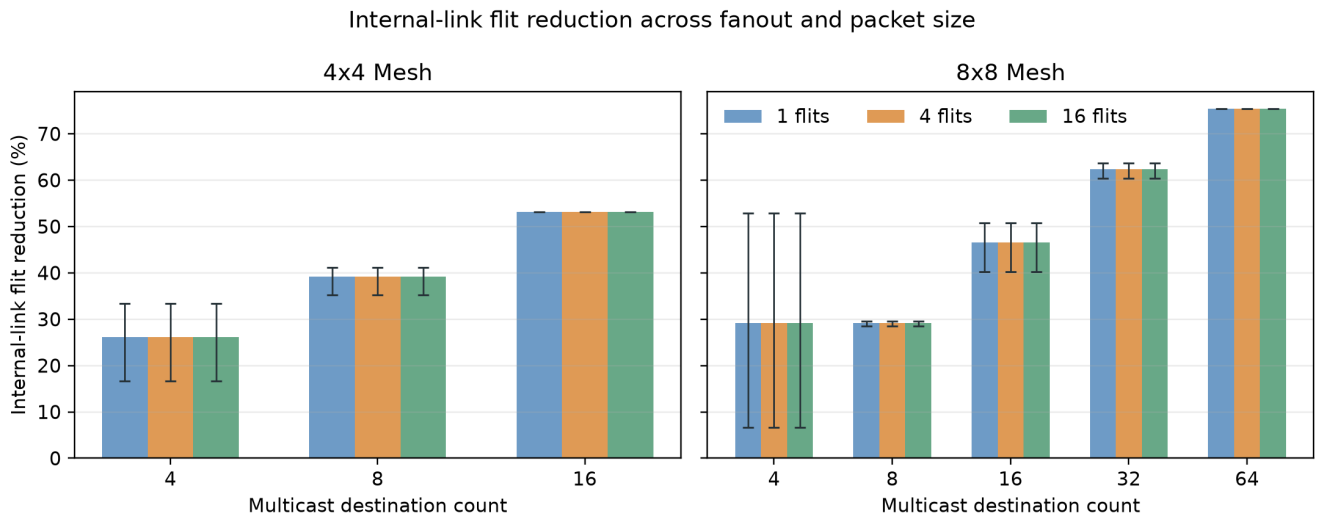


Figure 4: Mean internal-link flit reduction by topology, fanout, and packet size. Each bar averages matched offered-load, background-load, and seed cases; whiskers show observed ranges.

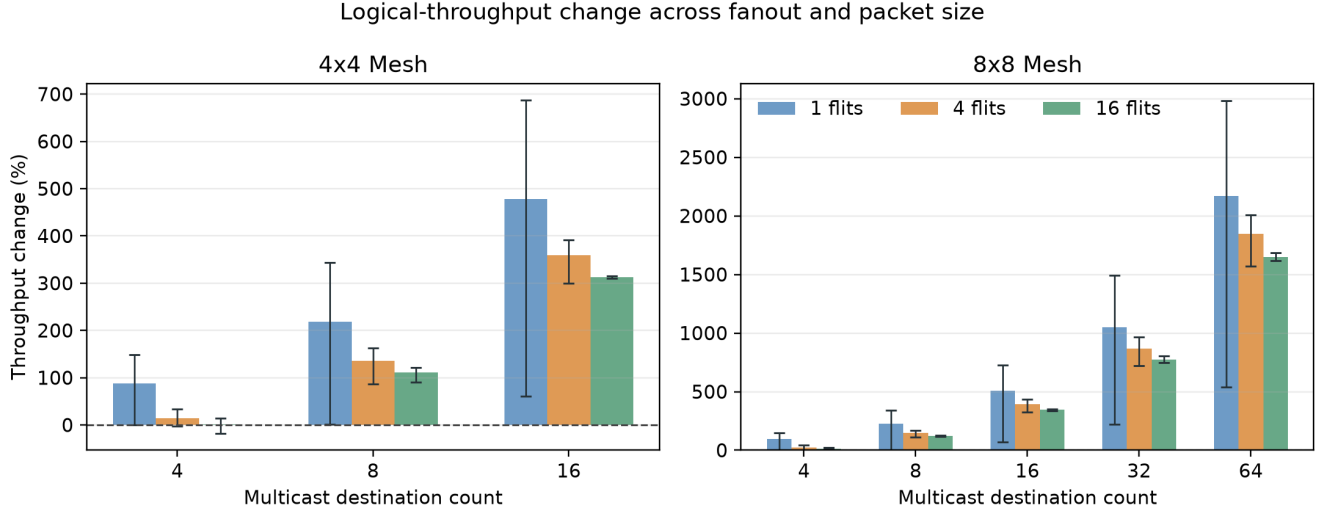


Figure 5: Multicast logical-throughput change by topology, fanout, and packet size. Bars are arithmetic means across matched offered-load, background-load, and seed cases; whiskers show observed ranges.

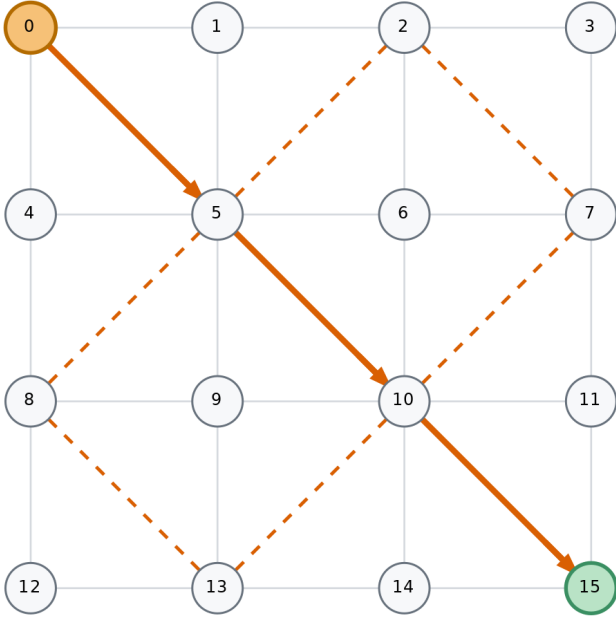
4 Pressure-aware express-link bypass

4.1 Proposed topology and baseline

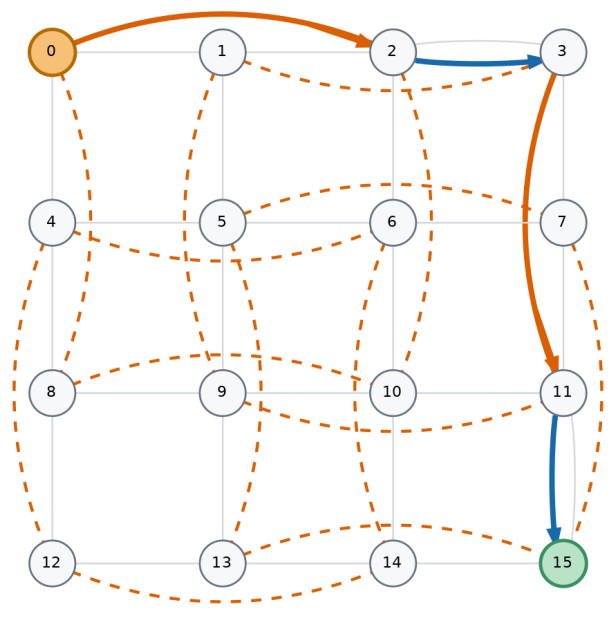
The two `Mesh_Bypass` families append bidirectional express links to the ordinary Mesh: a diagonal placement and a dimension-aligned stride-2 placement. Before simulation, a route-table builder enumerates every current-Router/destination pair and compares ordinary XY with each legal express edge over the complete remaining path. The static stride table repeats aligned express hops within X and then within Y; the static diagonal table uses an $X^* \rightarrow \text{diagonal}^* \rightarrow Y^*$ chain. An express entry is retained only when it lowers the modeled suffix cost, and exact-cost ties remain on ordinary XY.

The selected design uses a distance-scaled wire model: each express-link latency scales with its Manhattan span. An ablation also evaluated an optimistic model in which every express link costs one cycle; that model is treated only as an upper bound on the value of shortening a path. The plain Mesh XY comparator has fewer links, ports, VCs, and buffers, so this is a workload-matched, added-resource comparison rather than an iso-area, iso-power, or iso-wire claim.

Figure 6 contrasts the two placements.



(a) Phase-ordered diagonal alternative



(b) Proposed stride-2 topology

Figure 6: Illustrative 4×4 bypass placements and selected multi-hop routes. The diagonal alternative uses a phase-ordered $X^* \rightarrow \text{diagonal}^* \rightarrow Y^*$ chain; the primary stride design repeatedly selects aligned express links while preserving DOR.

Legend: orange dashed lines are installed bidirectional bypass links; solid orange arrows are selected express hops; solid blue is the ordinary DOR segment; gray lines are ordinary Mesh links.

4.2 Multi-hop routing and runtime admission

Topology construction creates two directed Garnet links for every installed physical shortcut and scales their latency by Manhattan span. The cost of an ordinary edge is link latency plus Router latency. An express edge uses its configured wire latency plus one Router traversal and is entered in the table only if it lowers the complete suffix cost. Listing 2 shows the stride-2 dynamic-programming recurrence; the diagonal builder applies the same complete-suffix comparison to its phase-ordered diagonal chain.

Listing 2: Source excerpt: cycle-aware multi-hop DOR selection (`bypass_oracle.py`).

```
@lru_cache(maxsize=None)
def best_stride_step(current, destination):
    if current == destination:
        return 0, None
    ordinary_next = xy_step(current, destination)
    suffix_cost, _ = best_stride_step(ordinary_next, destination)
    choices = [(ordinary_edge_cost + suffix_cost, 0,
                ordinary_next, "", None)]
    for link in outgoing[current]:
        if not dor_express(link, destination):
            continue
        suffix_cost, _ = best_stride_step(link["dst"], destination)
        cost = int(link["latency"]) + router_latency + suffix_cost
        choices.append((cost, 1, link["dst"], link["name"], link))
    cost, _, _, _, selected = min(choices)
    return cost, selected
```

At runtime, each Router looks up its precomputed current-Router/destination entry; it does not recompute the cycle-cost recurrence. Ordered Vnets retain that deterministic entry. For unordered traffic, the selector chooses between the table's express hop and monotonic XY using local free VCs and credits, the landing Router's escape/lookahead pressure, and destination-hotspot state. The destination-skew detector evaluates epochs of $4N$ data packets, enters the hot state at four times the uniform share, and retains it until the share falls below twice uniform.

Admission is also packet-aware. Diagonal 1-flit packets, and diagonal 4-flit packets on 4×4 , use at most one express hop before continuing on XY. Packets longer than 32 flits use the full conservative test: the express output must be at least as healthy as XY, the selected landing lookahead must be idle, and an entirely idle XY alternative remains on XY. These rules are part of the evaluated diagonal and stride-2 arms.

4.3 Routing safety and validation

The builder materializes every source–destination path, adds every successive channel pair to a dependency graph, and requires a topological ordering over the entire graph. A cycle formed across multiple routes therefore fails construction. Stride runtime adaptation can choose only the table’s express edge or monotonic XY; both strictly approach the destination, complete X before Y, and never reverse dimension. Because diagonal packet-aware fallback may leave the static diagonal chain early, the builder separately constructs and audits the complete union of its runtime express-or-XY choices rather than relying only on the static phase-ordered paths.

Route/dependency, traversal, and multi-flit/backpressure suites pass 5, 8, and 14 cases, respectively. Each bypass family contains 3,072 runs, or 1,536 matched pairs. Analysis checks pair completeness, finite metrics, route metadata, and run counts before aggregation.

4.4 Experimental setup

Each pair uses the same traffic pattern, packet size, offered load, seed, and pre-generated source/destination counts on plain Mesh XY and Mesh Bypass. The 4×4 and 8×8 matrices use uniform random, transpose, bit complement, and 50% hotspot traffic; 1/4/16/64-flit packets; 16 offered-flit points from 0.01 to 3.0 per node/source cycle; and seeds 1, 7, and 17. Each run uses 1,000 warm-up, 100,000 drain, 5,000 measurement, and 1,000,000 cooldown source cycles, giving 768 matched pairs per Mesh size and 1,536 per bypass family. Both evaluated arms use data VNet 2 with four buffers per VC and distance-scaled wires.

Figure 7 gives an illustrative 4×4 view of the four traffic patterns used in both Mesh sizes. The pattern definitions, not the particular 4×4 drawing, are what remain fixed across the paired sweep: transpose creates a diagonal permutation, bit complement crosses the mesh, uniform random spreads arrivals, and hotspot concentrates half of the packets on one Router.

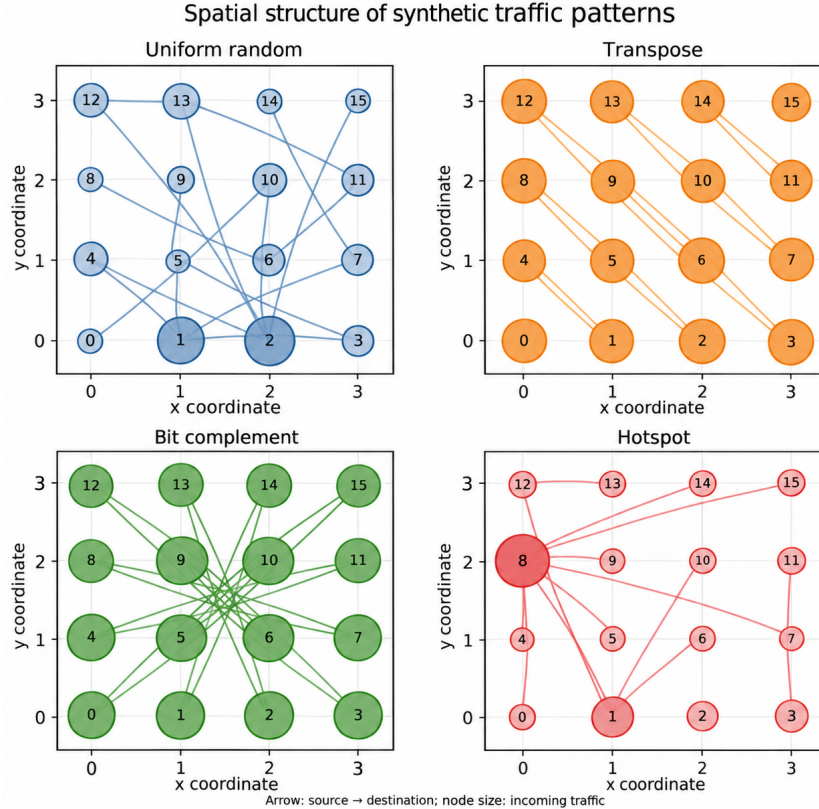


Figure 7: Illustrative spatial structure of the four synthetic traffic patterns on a 4×4 Mesh. Arrows show source-to-destination mappings and node size shows incoming traffic; the same definitions are evaluated on both Mesh sizes.

4.5 Bypass results

The two distance-scaled bypass families are summarized in one table and a latency breakdown by traffic pattern. Each family covers two Mesh sizes, four traffic patterns, four packet sizes, 16 offered loads, and three seeds: 1,536 matched

cases and 3,072 gem5 runs. Table 2 reports the aggregate results.

Table 2: Aggregate distance-scaled bypass results. For latency, each packet-size value is the geometric mean over offered load and seed; the four packet-size values are averaged arithmetically within each traffic pattern, and the four traffic-pattern values are combined geometrically. Throughput is the arithmetic mean over all matched cases.

Topology	Mesh	Latency	Throughput
Diagonal	4×4	1.426×	+4.27%
Diagonal	8×8	1.165×	+12.90%
Stride-2	4×4	1.312×	+4.37%
Stride-2	8×8	1.620×	+39.37%

Figure 8 breaks latency speedup down by traffic pattern, placement, and Mesh size.

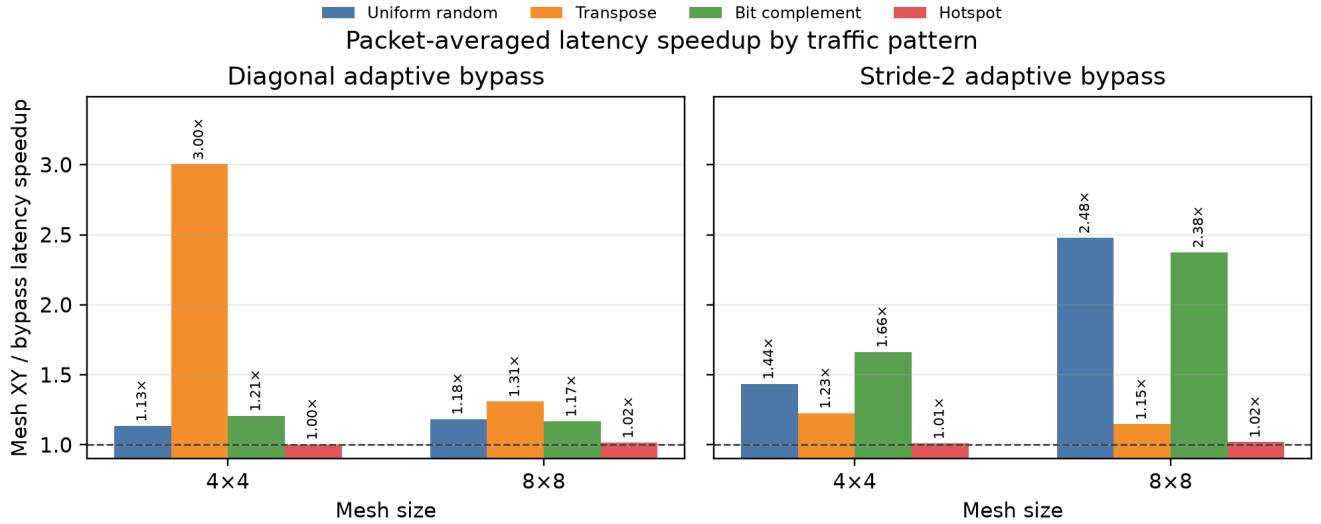


Figure 8: Packet-averaged latency speedup by traffic pattern for the distance-scaled diagonal and stride-2 adaptive bypass families. Values above 1× finish before plain Mesh XY.

Figure 9 uses the same grouping for throughput improvement.

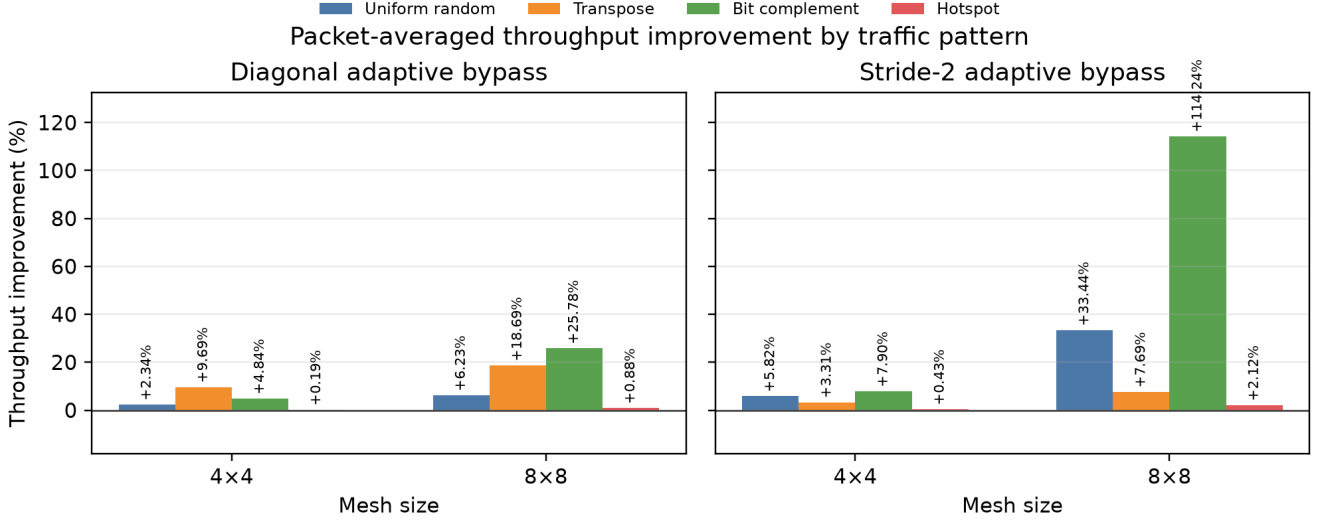


Figure 9: Packet-averaged throughput improvement by traffic pattern for the distance-scaled diagonal and stride-2 adaptive bypass families. Each value is the arithmetic mean over packet size, offered load, and seed for that traffic pattern and Mesh size.

4.6 Analysis across traffic patterns

Figures 8 and 9 expose whether a placement matches the geometry of a traffic pattern. For each latency bar, load and seed are first combined geometrically within each packet size and the four packet-size values are then averaged arithmetically. Each throughput bar is the arithmetic mean over packet size, offered load, and seed. The two figures therefore compare the same matched cases but measure different effects: completion time for latency and sustained completed-flit rate for throughput.

Uniform random. Stride-2 scales more strongly than diagonal. From 4×4 to 8×8 , its latency speedup rises from $1.436\times$ to $2.478\times$ and throughput improvement from $+5.82\%$ to $+33.44\%$; diagonal reaches only $1.135\times/1.184\times$ and $+2.34\%/+6.23\%$. Random pairs do not consistently present equal X/Y displacement for a diagonal phase, but many longer 8×8 routes contain repeated axis-aligned two-hop segments. Stride-2 can use those segments in both dimensions while retaining monotonic X-before-Y progress, so the opportunity grows with Mesh diameter.

Transpose. Diagonal is the natural match because transposition exchanges coordinates and frequently creates paired X/Y displacement. It produces the largest diagonal latency result, $3.004\times$ on 4×4 , compared with $1.227\times$ for stride-2; their throughput improvements are $+9.69\%$ and $+3.31\%$. On 8×8 , diagonal remains ahead at $1.310\times$ and $+18.69\%$, versus $1.147\times$ and $+7.69\%$ for stride-2. The latency gain does not scale as quickly as Mesh size because diagonal wires use distance-scaled latency and the phase-ordered route can exploit a diagonal hop only while both coordinate directions remain compatible.

Bit complement. Complement traffic creates long paths across both dimensions, favoring repeated stride hops. Stride-2 improves from $1.663\times/+7.90\%$ on 4×4 to $2.376\times/+114.24\%$ on 8×8 . Diagonal reaches $1.208\times/+4.84\%$ and $1.169\times/+25.78\%$, respectively. The larger Mesh supplies more legal stride-2 entries along a route, reducing Router traversals repeatedly rather than relying on a single geometric alignment. Because bit-complement destinations remain distributed, these savings also relieve network contention instead of merely moving it to one landing Router.

Hotspot. All four topology/size combinations remain close to Mesh XY: latency speedup ranges from $1.004\times$ to $1.020\times$, and throughput improvement from $+0.19\%$ to $+2.12\%$. Half of the traffic converges on one destination, so the last links, landing Router, and destination service rate remain shared bottlenecks regardless of the earlier shortcut. Runtime admission responds to this structure directly: landing pressure and hotspot state suppress an express hop when it would deliver packets faster into the same congested endpoint.

Across the four patterns, diagonal is therefore specialized: its clearest latency advantage appears when traffic follows its geometric phase. Stride-2 is broader and scales with path length, which explains why its 8×8 aggregate reaches $1.620\times$ latency speedup and $+39.37\%$ throughput. The figures also show why latency and throughput must both be reported. A shortcut can shorten an individual route, as in 4×4 transpose, while throughput remains bounded by shared outputs; when shortcuts are reusable and destinations remain distributed, as in 8×8 bit complement, both completion time and sustained rate improve strongly.

5 Trace-driven H100 tensor all-reduce (Topic 4 extension)

The synthetic experiments above isolate network mechanisms; this trace-based Topic 4 study reconnects them to the GPU training loop in Figure 1. It preserves the ordering and tensor sizes of measured collectives while comparing scalar-lane and event-level request representations. In this way, the case study extends the multicast question from one-to-many delivery to the reduce-and-broadcast structure of all-reduce.

5.1 Tensor datapath implementation

Tensor packets retain their multi-flit boundary while each flit carries a lane identifier. A collective flit is intercepted before ordinary input-VC insertion, route computation, switch allocation, and crossbar traversal. The Router collective state machine accumulates each (collective, lane) key and forwards the completed value through a queued reduce/broadcast path. Completed lanes wait when the next output VC is unavailable, so backpressure does not discard reduction state.

Listing 3: Abridged source excerpt: lane-wise tensor accumulation (`Router.cc`).

```
const int lane = t_flit->get_lane_id();
const auto key = std::make_pair(t_flit->get_collective_id(), lane);
CollectiveLaneState& state = m_collective_lanes[key];
state.sum += t_flit->get_value();
++state.count;
if (state.count == m_collective_expected_fanin) {
    const int64_t sum = state.sum;
    m_collective_lanes.erase(key);
    if (m_collective_root)
        forwardCollectiveLane(sum, lane, CollectiveOp::Broadcast,
                               m_id, t_flit);
    else
        forwardCollectiveLane(sum, lane, CollectiveOp::Reduce,
                               parent, t_flit);
}
```

5.2 Trace lowering and experimental setup

The input comes from an eight-H100 80GB HBM3 NCCL microbenchmark (PyTorch 2.8.0+cu128, CUDA 12.8, NCCL 2.27.3). The replay compiler selects 15 measurement-phase broadcasts and 15 all-reduces: five repetitions each at 1, 4, and 16 MiB. It represents communication with 16-byte flits and scales payload bytes and relative release times by 1/1024. An all-reduce event is therefore 1, 4, or 16 KiB in the replay, corresponding to 64, 256, or 1,024 flits per rank and 6,720 lanes per rank across all 15 events. Scaling preserves event ordering and produces a tractable Garnet replay; the reported times are simulation ticks rather than native H100 or NVLink timings.

For broadcast, the compiler splits events into variable-length tree-multicast packets. For all-reduce, scalar lowering repeats each event’s release cycle for N independent one-flit collective rounds. Tensor lowering instead creates one N -flit packet per event and rank, with a lane ID on every flit. Both execute the same lane-wise Router reduction followed by tree broadcast. The JSON records chunks of at most 4 KiB after scaling. The Garnet loader combines these chunks into each event’s total `tensor_flits` and executes one logical tensor request per event. The scalar and tensor paths are distinct from bitmap-based `tree_multicast`, although all three use shared tree prefixes.

Figure 10 connects the exact trace sizes, the two lowerings, the Router operation, and the two controlled outcomes.

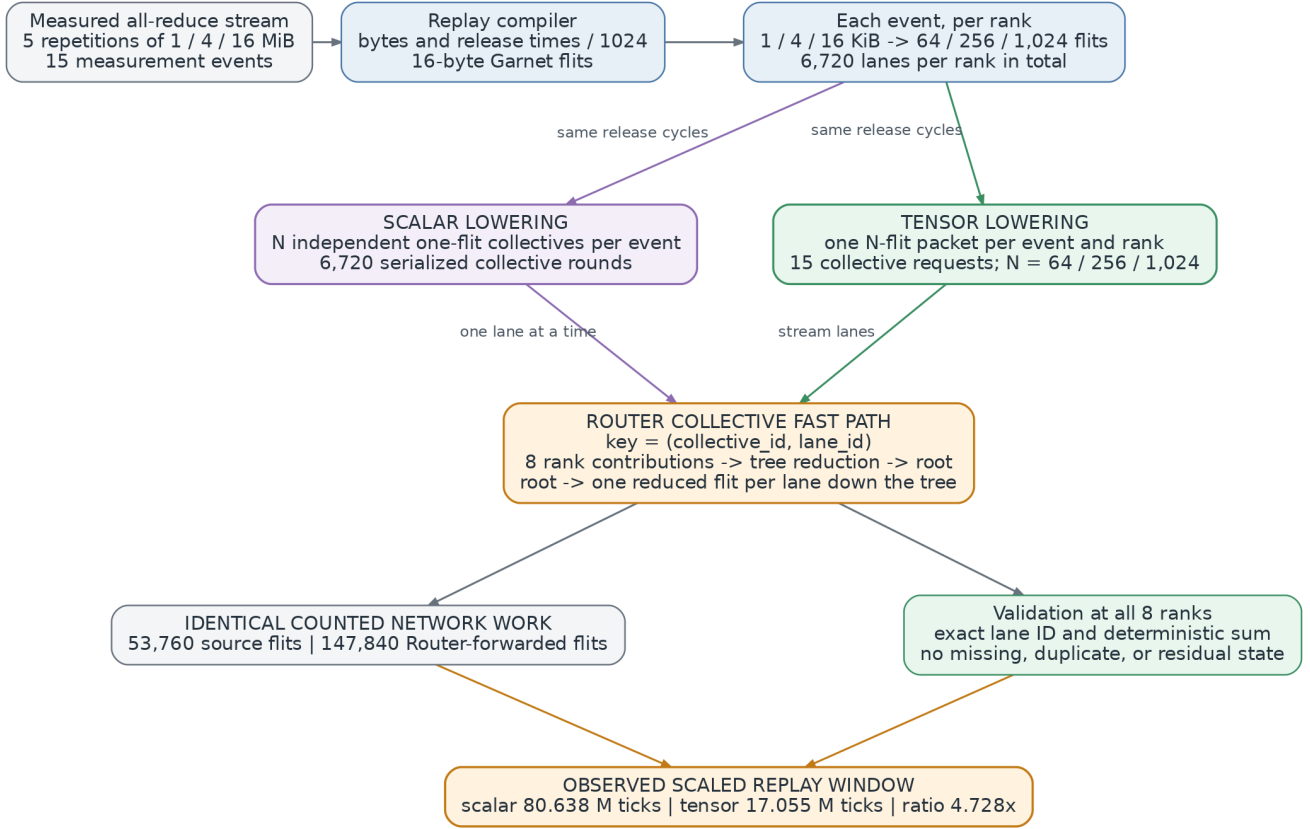


Figure 10: Scalar and tensor lowering of the same scaled all-reduce events. The manipulated variable is packet/request representation; release cycles, lane arithmetic, routes, and counted network work are held fixed.

The reported replay runs on Mesh XY so that the Topic 4 comparison isolates the request representation from topology changes.

5.3 Correctness and backpressure validation

Fourteen functional cases verify exact lane counts and reduced values across varied meshes, and 32 backpressure cases vary link latency, Router latency, and occupancy while requiring completion without residual accumulator state. Twelve trace-tool unit tests check schema, event sizes and order, collective IDs, and participants. In the evaluated replay, both all-reduce representations complete the same 53,760 rank-lane deliveries and 147,840 Router-forwarded flits; every lane ID and deterministic sum matches.

5.4 Replay experiments and results

The broadcast compiler expands 15 source events into 30 tree-multicast packets because each scaled 16-KiB event is represented by four 4-KiB packets. The Mesh-XY replay completes 6,720 source flits and 210 request-destination deliveries in a 17,047,500-tick window. The all-reduce comparison holds the route and scaled release schedule fixed while changing scalar-lane versus event-level packetization.

Table 3 records the three Mesh-XY replay representations.

Table 3: Scaled H100 replay summary. Logical requests are collective-level operations; source flits are network injections summed across participating ranks.

Replay view	Trace events	Logical requests	Source flits	Window (ticks)
Broadcast	15	30	6,720	17,047,500
Scalar all-reduce	15	6,720	53,760	80,638,000
Tensor all-reduce	15	15	53,760	17,054,500

All accumulator entries are released at completion. Event-level tensor packetization reduces the replay window in Table 3 from 80,638,000 to 17,054,500 ticks, a $4.728\times$ improvement on Mesh XY. Since the two representations inject and forward the same number of flits, the difference comes from replacing 6,720 independently admitted scalar-lane rounds with 15 event-level collective requests. The experiment therefore measures the scheduling effect of request representation in the collective datapath.

6 Division of Labor

Chunyu Liu:

- Stride-2 Express-Link Bypass Design and Evaluation;
- Tree Multicast Design and Evaluation;
- H100 Trace Capture and Compilation;
- Report and Slide Production;
- On-Site Presentation.

Boyan Pu:

- Diagonal Express-Link Bypass Design and Evaluation;
- Tensor All-Reduce Implementation;
- Backpressure and Correctness Validation;
- Report and Slide Production.

7 Reproducibility

The evaluated multicast/tensor implementation is revision `77fcf26d57`. The multi-hop bypass evaluation records source revision `1e8764cde7`, with source hashes matching the pressure-aware implementation at `ea4c3c9b0f`. Machine-readable manifests record software versions, exact arguments, measurement windows, source hashes, and raw-result locations. The analysis scripts verify counts, finite metrics, pair completeness, and VNet/routing metadata before producing summaries.

Report figures can be rebuilt from committed inputs with:

```
python3 report/scripts/plot_multicast.py
python3 report/scripts/plot_bypass_topology.py
python3 report/scripts/generate_architecture_figures.py
```

The validation runner `run_lab4_full_gate.py` executes seven functional, backpressure, and trace suites. The proposed bypass campaign is reproduced separately from the full functional gate.

8 Conclusion

The implementation satisfies both microarchitecture requirements in Lab 4 Topic 3: physical links bypass intermediate Routers, and Router-side replication sends one packet to multiple destinations. Each mechanism is evaluated against an explicit baseline under matched workloads. On 4×4 , tree multicast removes 39.51% of internal-link flits and achieves $3.638\times$ geometric-mean latency speedup across 162 matched cases; its ten throughput regressions all occur at fanout 4. On 8×8 , the common-fanout matrix removes 34.95% of link traffic, achieves $3.472\times$ latency speedup, and has no throughput regressions across another 162 cases. The separate 8×8 scale-out study reaches 75.39% traffic reduction at fanout 64. This separation identifies atomic branch synchronization as a 4×4 low-fanout effect rather than a general multicast trend. The latency results reflect both shared-prefix replication and the dedicated Router datapath. The bypass design builds a complete-suffix cycle-cost table offline; each Router looks up that table and admits distance-scaled express hops under packet-aware local, landing, and hotspot pressure rules. The aggregate bypass result is $1.426\times/+4.27\%$ for diagonal on 4×4 , $1.165\times/+12.90\%$ for diagonal on 8×8 , $1.312\times/+4.37\%$ for stride-2 on 4×4 , and $1.620\times/+39.37\%$ for stride-2 on 8×8 . Mesh_Bypass adds links, ports, and buffers, making this

an added-resource comparison with explicit topology-specific cost proxies. The H100 trace then extends the collective study to complete tensors: preserving 15 event-level multi-flit all-reduce requests shortens the scaled replay window by $4.728\times$ with identical counted network work.

Together, the experiments show how exposing communication structure to the network removes duplicated transport and unnecessary request scheduling. Tree replication shares multicast prefixes, event-level tensor packets avoid scalar-lane serialization, and pressure-aware routing directs express traffic according to modeled cycle cost and current congestion.

References

- [1] N. Binkert et al., “The gem5 Simulator,” *ACM SIGARCH Computer Architecture News*, vol. 39, no. 2, 2011, pp. 1–7. doi: [10.1145/2024716.2024718](https://doi.org/10.1145/2024716.2024718).
- [2] N. Agarwal, T. Krishna, L.-S. Peh, and N. K. Jha, “GARNET: A Detailed On-Chip Network Model inside a Full-System Simulator,” *ISPASS*, 2009, pp. 33–42. doi: [10.1109/ISPASS.2009.4919636](https://doi.org/10.1109/ISPASS.2009.4919636).
- [3] N. E. Jerger, L.-S. Peh, and M. H. Lipasti, “Virtual Circuit Tree Multicasting: A Case for On-Chip Hardware Multicast Support,” *ISCA*, 2008, pp. 229–240. doi: [10.1109/ISCA.2008.12](https://doi.org/10.1109/ISCA.2008.12).
- [4] A. Kumar, L.-S. Peh, P. Kundu, and N. K. Jha, “Express Virtual Channels: Towards the Ideal Interconnection Fabric,” *ISCA*, 2007, pp. 150–161. doi: [10.1145/1250662.1250681](https://doi.org/10.1145/1250662.1250681).
- [5] W. J. Dally and C. L. Seitz, “Deadlock-Free Message Routing in Multiprocessor Interconnection Networks,” *IEEE Transactions on Computers*, vol. 36, no. 5, 1987, pp. 547–553. doi: [10.1109/TC.1987.1676939](https://doi.org/10.1109/TC.1987.1676939).
- [6] R. L. Graham et al., “Scalable Hierarchical Aggregation and Reduction Protocol (SHARP) Streaming-Aggregation Hardware Design and Evaluation,” *High Performance Computing*, 2020. <https://pmc.ncbi.nlm.nih.gov/articles/PMC7295336/>.
- [7] NVIDIA, “NVIDIA Collective Communication Library Documentation.” <https://docs.nvidia.com/deeplearning/nccl/user-guide/>.